

# YANFLÈ — Modèles disponibles

## Modèles disponibles

Ce dossier contient les poids utilisés par le backend (`../backend/main.py`). Le système ne charge plus un seul modèle mais une **chaîne de modèles**, listée dans `MODELS_INFO (main.py)` : chaque caméra fait tourner tous les modèles de la chaîne en parallèle sur chaque image, et leurs résultats sont combinés pour la narration.

Tous les modèles tournent sur GPU via CUDA quand disponible. Les temps indiqués sont mesurés “à chaud” (après les premiers appels de chauffe du GPU), sur cette machine — à titre indicatif, pas une garantie sur un autre matériel.

### yolo11n.pt — détection, nano

- **Fait** : détecte 80 classes d’objets courants (COCO) — personnes, véhicules, animaux, objets du quotidien. Boîte englobante + classe + confiance.
- **Poids** : 5,4 Mo
- **Vitesse** : ~7-13 ms/image
- **Usage** : le plus rapide, pour prototyper vite ou tourner sur une machine sans GPU puissant. Précision correcte mais la plus faible des trois tailles de détection classique.

### yolo11m.pt — détection, medium

- **Fait** : identique au nano (80 classes COCO), mais réseau plus profond → détections plus précises, surtout sur les petits objets ou scènes chargées.
- **Poids** : 39 Mo
- **Vitesse** : ~10-11 ms/image
- **Usage** : bon compromis précision/vitesse quand le GPU suit.

### yolo11x.pt — détection, extra-large

- **Fait** : identique au nano/medium (80 classes COCO), version la plus précise de la famille détection classique.
- **Poids** : 110 Mo
- **Vitesse** : ~26-28 ms/image
- **Usage** : quand la précision prime sur la vitesse ; reste largement temps réel sur RTX 4060.

### yolo11x-pose.pt — pose + tracking

- **Fait** : ne détecte que la classe **personne**, mais donne en plus :

- un **squelette** par personne (17 points clés : tête, épaules, coudes, poignets, hanches, genoux, chevilles)
- un **track\_id** stable dans le temps (via `model.track(..., persist=True)`, tracker ByteTrack) — permet de savoir depuis combien de temps une personne donnée est présente dans le champ de la caméra
- **Poids** : 113 Mo
- **Vitesse** : ~38 ms/image (un peu plus lourd, calcule le squelette en plus)
- **Usage** : brique de base pour analyser un **comportement** (posture, présence prolongée) plutôt qu’une simple présence — c’est l’axe “vigie comportementale” du projet YANFLÈ.
- **Limite** : ne voit plus les autres objets (bus, sac, couteau...), juste des personnes.

### threat-yolov8n.pt — détection de menaces

- **Fait** : détecte 4 classes liées à une menace potentielle : **Gun** (arme à feu), **explosion**, **grenade**, **knife** (couteau).
- **Poids** : 6,3 Mo
- **Vitesse** : ~13-16 ms/image à chaud (comme les autres modèles nano)
- **Source** : modèle communautaire fine-tuné, publié sur Hugging Face ([Subh775/Threat-Detection-YOLOv8n](#)), licence MIT. Entraîné sur un jeu de données Roboflow, **pas un modèle officiel Ultralytics**.
- **Précision annoncée** : 81,3% mAP@50 sur la validation de son propre jeu de test — correcte mais nettement en dessous des modèles officiels YOLO11 (90%+). À prendre avec prudence : plus de faux positifs/négatifs attendus, notamment hors du contexte des images d’entraînement.
- **Usage** : détecter des objets dangereux dans le champ de la caméra. Ne détecte **pas** les personnes — pour savoir qui tient l’objet détecté, il faudrait le combiner avec un modèle de personnes/pose (ex. `yolo11x-pose.pt`) et une règle de proximité (boîte de l’arme proche du poignet d’une personne).

### fire-yolo11s.pt — détection de feu/fumée

- **Fait** : repère un départ de feu ou de la fumée dans le champ de la caméra.
- **Poids** : 18,3 Mo
- **Vitesse** : ~4-5 ms/image (GPU, mesuré sur cette machine)
- **Source** : modèle communautaire, non officiel Ultralytics — provenance exacte non re-documentée au moment de l’ajout au projet. À revalider sur des cas réels avant un usage critique.
- **Usage** : rôle “Le Guetteur” dans la chaîne (`main.py`).

### plate-yolov11n.pt — plaques d’immatriculation

- **Fait** : repère les plaques d’immatriculation des véhicules.
- **Poids** : 5,2 Mo
- **Vitesse** : ~4-5 ms/image (GPU, mesuré sur cette machine)
- **Source** : modèle communautaire, non officiel Ultralytics — provenance exacte non re-documentée au moment de l’ajout au projet.
- **Usage** : rôle “Le Greffier” dans la chaîne.

### fight-yolov8s.pt — bagarre / violence

- **Fait** : distingue un comportement violent d'un comportement neutre (2 classes : violence / non-violence — seule la classe “violence” est gardée dans la chaîne via "classes": [1]).
- **Poids** : 21,5 Mo
- **Vitesse** : ~4-5 ms/image (GPU, mesuré sur cette machine)
- **Source** : modèle communautaire, non officiel Ultralytics — provenance exacte non re-documentée au moment de l'ajout au projet.
- **Usage** : rôle “L'Arbitre” dans la chaîne.

### yolo11n-seg.pt — segmentation

- **Fait** : identique à yolo11n (80 classes COCO), mais donne le contour précis (masque pixel par pixel) de chaque objet plutôt qu'une simple boîte englobante.
- **Poids** : 5,9 Mo
- **Vitesse** : ~5 ms/image (GPU, mesuré sur cette machine)
- **Source** : modèle officiel Ultralytics (COCO).
- **Usage** : rôle “Le Silhouette” dans la chaîne.

### facenet-vggface2.pt — reconnaissance faciale

- **Fait** : n'est pas un modèle YOLO. InceptionResnetV1 (facenet-pytorch) pré-entraîné sur VGGFace2, utilisé pour extraire un vecteur (embedding) par visage détecté par MTCNN. Sert à reconnaître si un visage a déjà été vu (mémoire à deux niveaux : candidat → personne confirmée), pas à identifier une personne à partir d'une base externe.
- **Poids** : 106,7 Mo
- **Source** : poids officiels VGGFace2 du projet facenet-pytorch, licence MIT.
- **Usage** : face\_memory.py, indépendant de la chaîne MODELS\_INFO.

### ppe-yolov8s.pt — équipements de protection (PPE)

- **Fait** : détecte le port (ou l'absence) d'équipements de sécurité : Hardhat / NO-Hardhat (casque), Safety Vest / NO-Safety Vest (gilet), Mask / NO-Mask, plus Person, Safety Cone, machinery, vehicle.
- **Poids** : 21,5 Mo
- **Vitesse** : ~4-5 ms/image (GPU, mesuré sur cette machine)
- **Source** : [VoxDroid/Construction-Site-Safety-PPE-Detection](#), licence MIT. YOLOv8s entraîné 200 epochs sur un jeu Roboflow “Construction Site Safety”.
- **Précision annoncée** : 87,7% mAP@50 sur la validation de son propre jeu de test.
- **Statut** : branché à la chaîne, rôle “L'Inspecteur” dans MODELS\_INFO. Seules les classes d'équipement sont gardées (classes: [0,1,2,3,4,6,7]) : Person/machinery/vehicle sont exclues pour ne pas doubler ce que les autres modèles de la chaîne détectent déjà.

### clothing-yolov8s-seg.pt — type de vêtements

- **Fait** : détecte et segmente 13 types de vêtements portés par une personne : chemise (manche courte/longue), veste (manche courte/longue), gilet, sling, short, pantalon, jupe, robe (manche courte/longue), robe gilet, robe sling.
- **Poids** : 22,7 Mo
- **Vitesse** : ~5-6 ms/image (GPU, mesuré sur cette machine)

- **Source** : [Bingsu/adetailer](#) (modèle `deepfashion2_yolov8s-seg.pt`), licence Apache-2.0. YOLOv8s-seg entraîné sur le jeu DeepFashion2, largement utilisé dans l'écosystème Stable Diffusion (projet ADetailer).
- **Précision annoncée** : 84,9% mAP@50 bbox / 76,3% mAP@50-95 bbox.
- **Statut** : branché à la chaîne, rôle “Le Couturier” dans `MODELS_INFO`. La couleur ajoutée entre parenthèses n'est pas une classe du modèle : elle est calculée par `colors_fr.py` sur les pixels réels de chaque boîte détectée (couleur médiane en HSV), donc toujours vérifiable, jamais devinée.

### gender-yolov8n-cls.pt — genre (homme / femme)

- **Fait** : classifie un crop de personne déjà découpé (pas de boîte englobante — ce n'est pas un détecteur mais un classifieur). 2 classes : `female` / `male`.
- **Poids** : 2,8 Mo
- **Source** : [DhanushSGowda/yolov8n-gender-classification](#), licence MIT. Entraîné sur le “Gender Classification Dataset” (Kaggle, 47 009 images), 97,7% de précision annoncée sur sa validation.
- **Limite reconnue par l'auteur lui-même** : provenance des images du dataset source non documentée, biais possible du dataset d'origine (voir la note “Privacy & Ethical” du modèle).
- **Statut** : branché, mais **pas** comme les autres — pas de rôle dans `MODELS_INFO` (whole-frame), utilisé uniquement dans `person_memory.py` pour classifier le crop de chaque personne détectée par le modèle pose.
- **Seuil** : une classification sous 0.75 de confiance est ignorée (`GENDER_MIN_CONF` dans `person_memory.py`) plutôt que d'afficher un genre incertain comme s'il était sûr.

### La fusion “personne” (`person_memory.py`)

Pas un modèle de plus, mais une brique qui combine les résultats des modèles déjà en place pour construire une fiche par personne détectée :

- **Identité** : nom si le visage est reconnu (Le *Physionomiste*), sinon `track_id` de la posture (Le *Comportementaliste*) comme identité temporaire pour la durée de la session caméra.
- **Signalement** : type + couleur de vêtements (Le *Couturier*) et équipement (L'*Inspecteur*) dont la boîte tombe dans celle de la personne.
- **Genre** : via `gender-yolov8n-cls.pt` sur le crop de la personne.
- **Présence prolongée** : mesure depuis combien de temps la même identité (visage ou `track_id`) reste dans le même point sans en ressortir. Au-delà de 120s (`LOITER_SECONDS`), signalé comme “rôdeur potentiel” dans la narration. Le compteur repart de zéro si la personne n'est plus revue pendant plus de 15s (`STALE_SECONDS`) — on considère qu'elle est partie.
- **Limite honnête** : la continuité d'identité sans visage repose sur le `track_id` du modèle de pose, qui peut se perdre (occlusion, sortie du champ) — une vraie coupure de tracking repart à zéro, ce n'est pas une ré-identification par apparence après une coupure complète.

### Le labo de test (`../test_lab/`)

Un petit serveur FastAPI séparé (port 8010, HTTPS pour l'accès caméra) permet de tester un modèle avant de l'intégrer à la vraie chaîne : caméra en direct, image annotée et détections en français. A servi à valider `ppe-yolov8s.pt` et `clothing-yolov8s-seg.pt` avant leur intégration dans `MODELS_INFO` — reste utile pour les prochains modèles.

## **Pour ajouter un nouveau modèle à la chaîne**

Voir `../docs/GUIDE_FINETUNING.md` pour la marche à suivre complète (entraînement, validation, intégration dans `MODELS_INFO` et `labels_fr.py`).